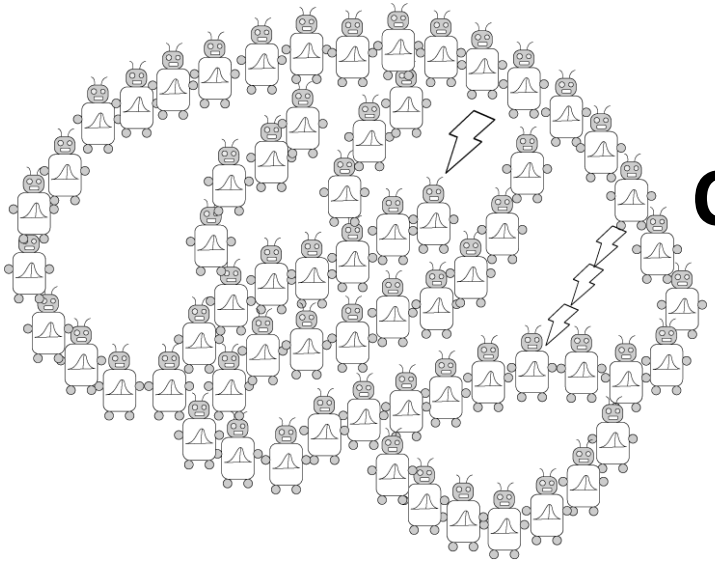
Introduction to Neural Networks



PSYCH/COMP SCI/BMI 841:
Computational Cognitive Science

Joseph Austerweil

April 17, 2025

- Admin Matters
- Connectionism and neural network basics
- Linear neural networks
- Nonlinearity and Backpropagation
- Convolutional and Recurrent neural networks

Short rest 1

- Three student presentations:
  1. Lake et al 2015 by Ruijia
  2. Linzen et al by Elijah

Short rest 2

3. Schulz Buschoff et al by Khailanii

Administrative Matters

- Assignment 3 was due April 9 at 8:59pm
- Assignment 4 is due April 23 at 8:59pm

- One more discussion post left
  - Only need to pass 5 through class for full credit

- Any unused quiz points will be free points to your final grade.

- Final project presentations May 1
  - Your project does NOT need to be complete by then
- Final project papers due May 6 at 8:59pm

- Questions?

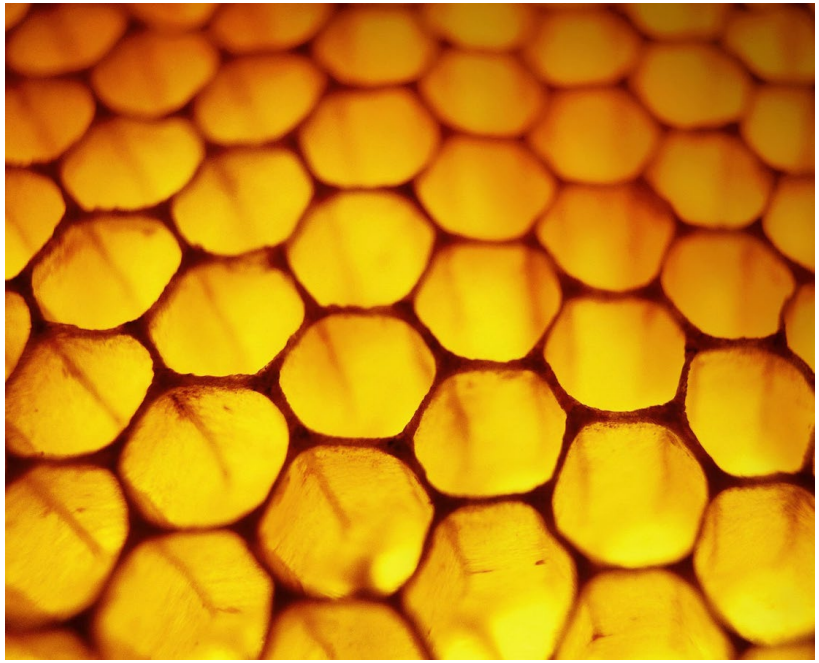
- Admin Matters
- Connectionism and neural network basics
- Linear neural networks
- Nonlinearity and Backpropagation
- Convolutional and Recurrent neural networks

Short rest 1

- Three student presentations:
  1. Lake et al 2015 by Ruijia
  2. Linzen et al by Elijah

Short rest 2

3. Schulz Buschoff et al by Khailanii



honeycomb

termite mound

Connectionism
(Parallel Distributed Processing)

Think about cognitive processes as the result of computations performed by a large number of interconnected simple units.

Activation of each unit depends on the activation of units with links to that unit.

Inspired by the way that neurons work.

- leads to models called “artificial neural networks.”

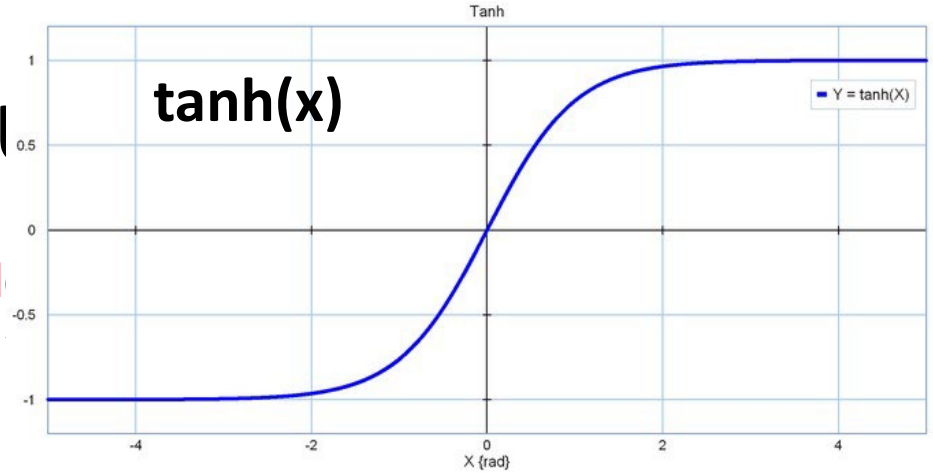
A caution on biological connections

One reason for the appeal of PDP models is their obvious "physiological" flavor: They seem so much more closely tied to the physiology of the brain than are other kinds of information-processing models. The brain consists of a large number of highly interconnected elements (Figure 3) which apparently send very simple excitatory and inhibitory messages to each other and update their excitations on the basis of these simple messages. The properties of the units in many of the PDP models we will be exploring were inspired by basic properties of the neural hardware. In a later section of this book, we will examine in some detail the relation between PDP models and the brain.

Though the appeal of PDP models is definitely enhanced by their physiological plausibility and neural inspiration, these are not the primary bases for their appeal to us. We are, after all, cognitive scientists, and PDP models appeal to us for psychological and computational reasons. They hold out the hope of offering computationally sufficient and psychologically accurate mechanistic accounts of the phenomena of human cognition which have eluded successful explication in conventional computational formalisms; and they have radically altered the way we think about the time-course of processing, the nature of representation, and the mechanisms of learning.

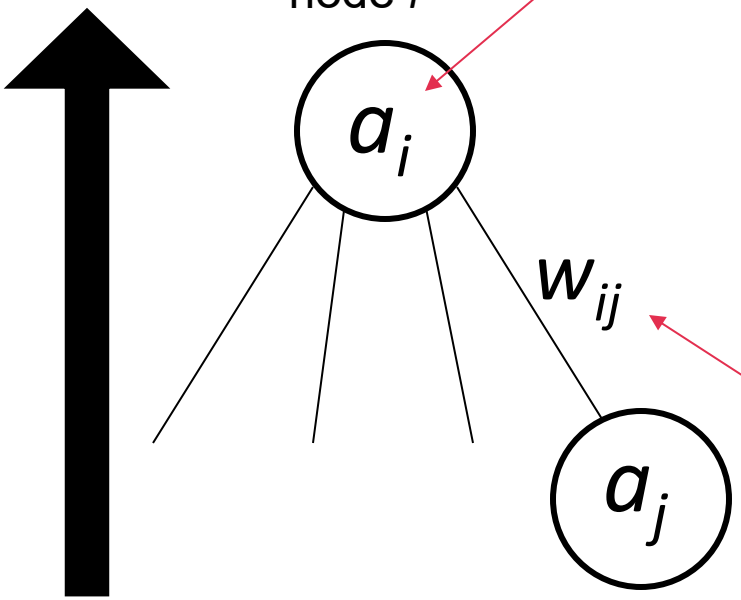
(McClelland, Rumelhart, & Hinton, 1986)

Artificial ne



activation of nod

node *i*



node *j*

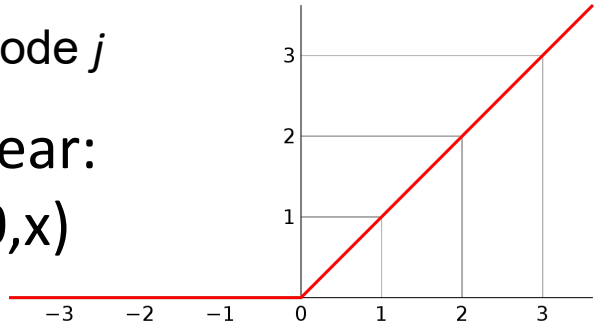
$$a_i = g(input_i)$$

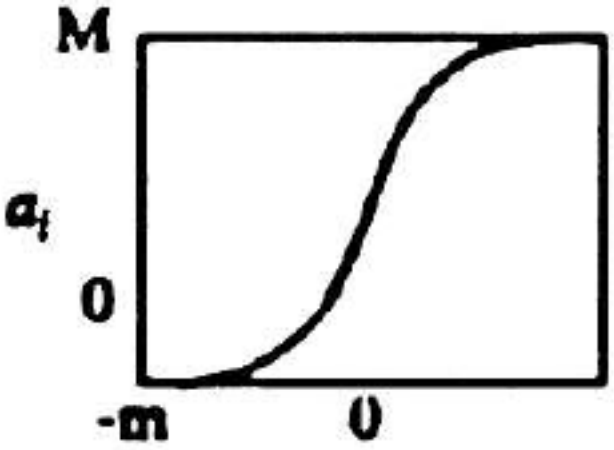
sum over *j*

$$input_i = \sum_j w_{ij} a_j$$

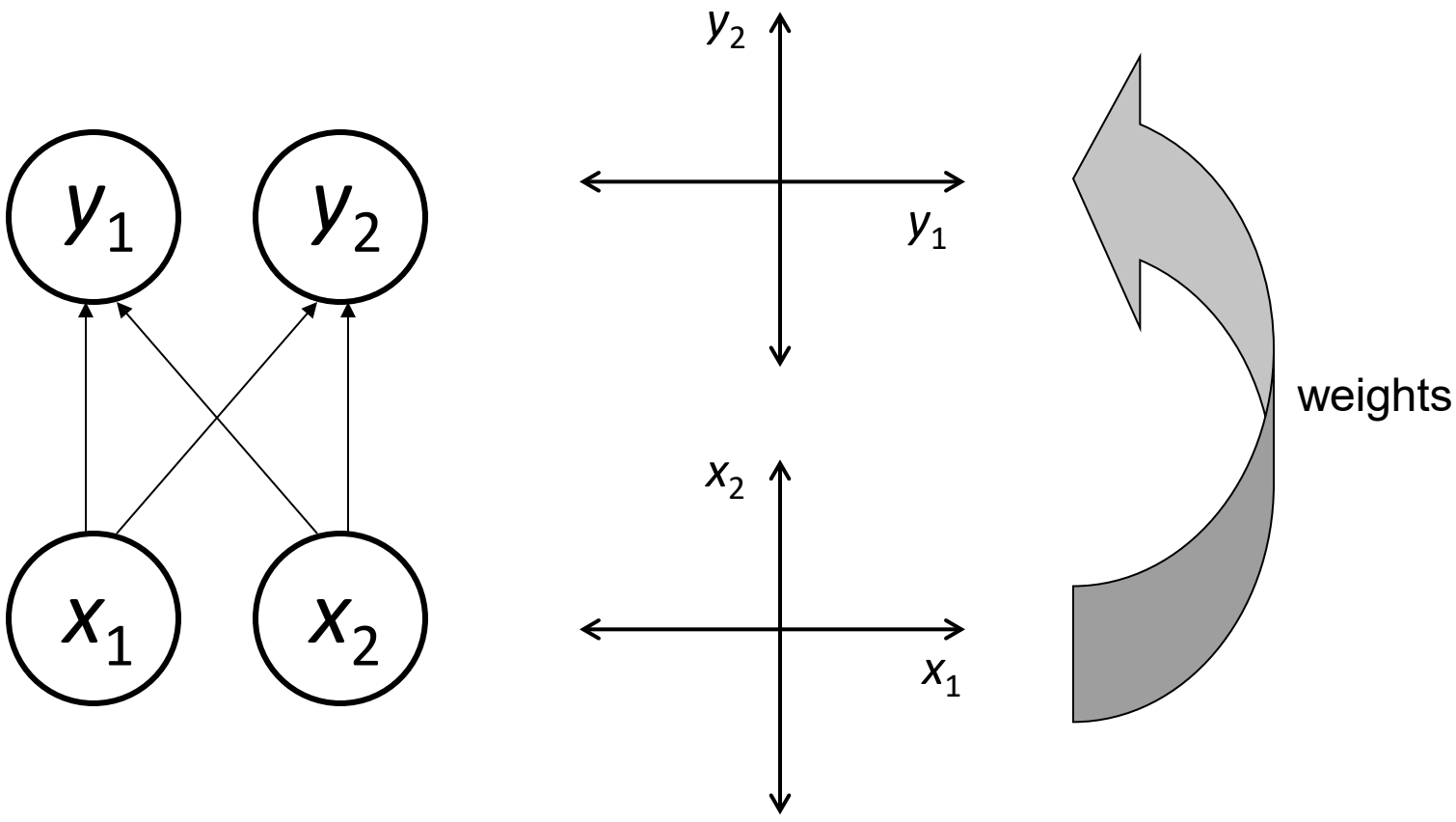
weight of link from *i* to *j*

Rectified linear:
g(x) = max(0,x)

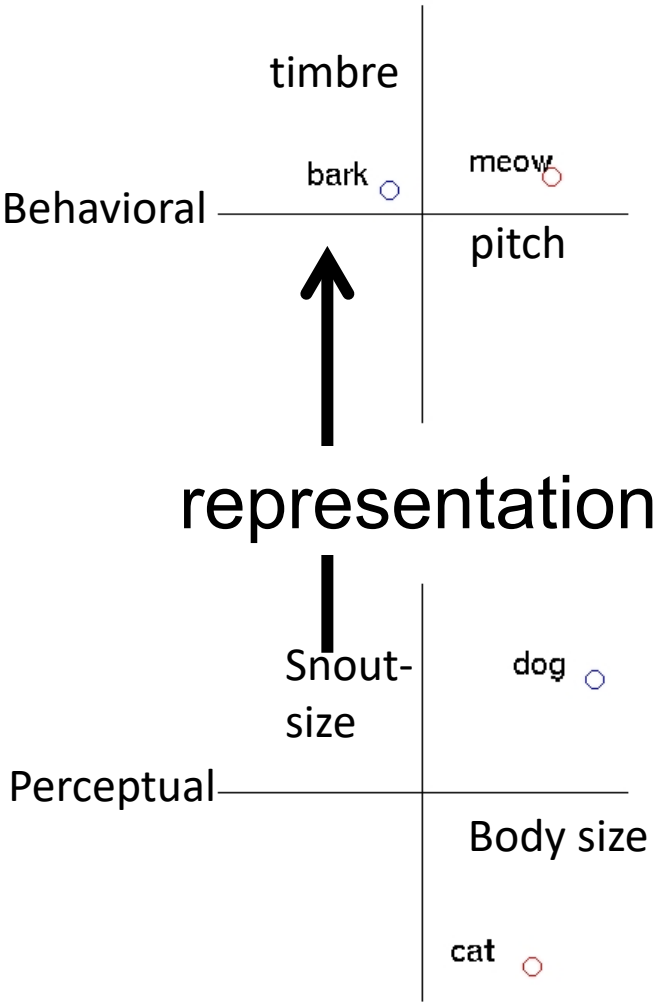




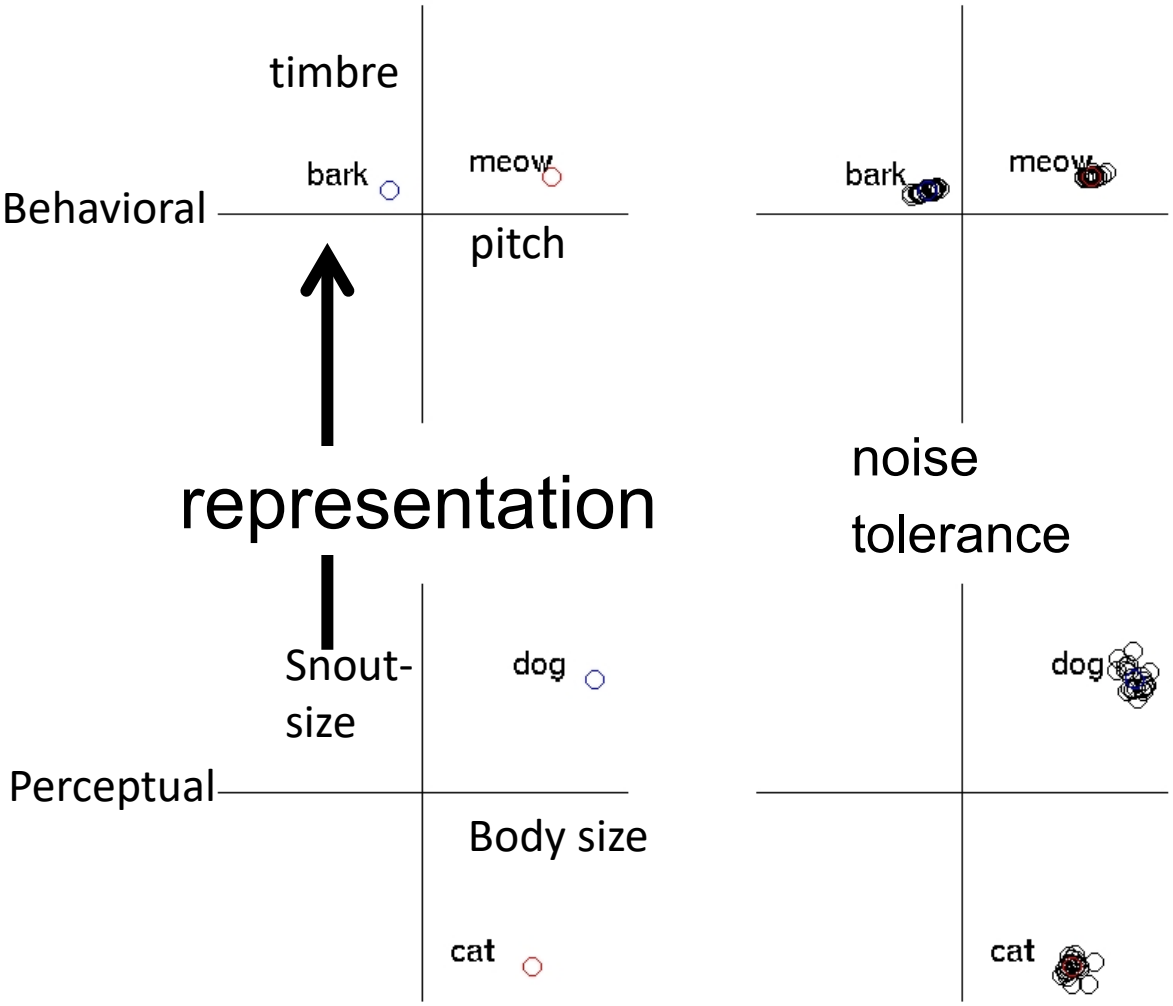
A very simple neural network



Distributed representations

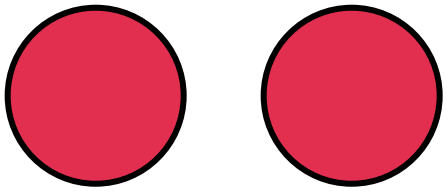


Distributed representations

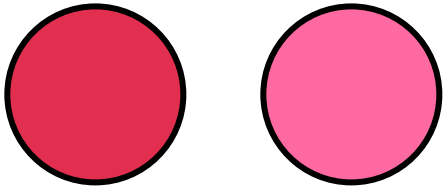


Different representations

dog

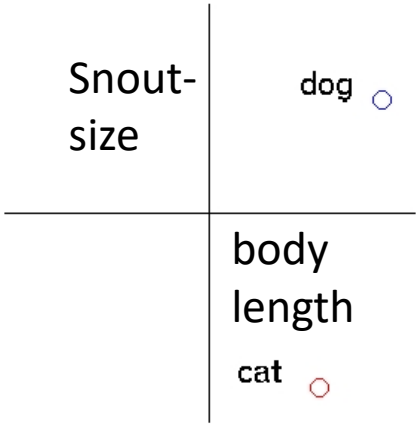


cat



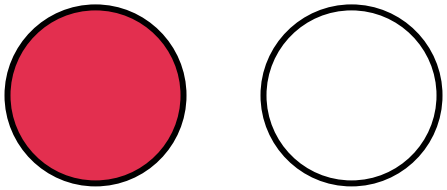
Distributed

One pattern of activation per concept

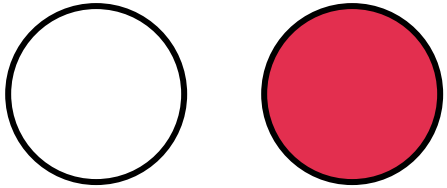


Interpretable as points in space

dog



cat



Localist

One node per concept

- Admin Matters
- Connectionism and neural network basics
- Linear neural networks
- Nonlinearity and Backpropagation
- Convolutional and Recurrent neural networks

Short rest 1

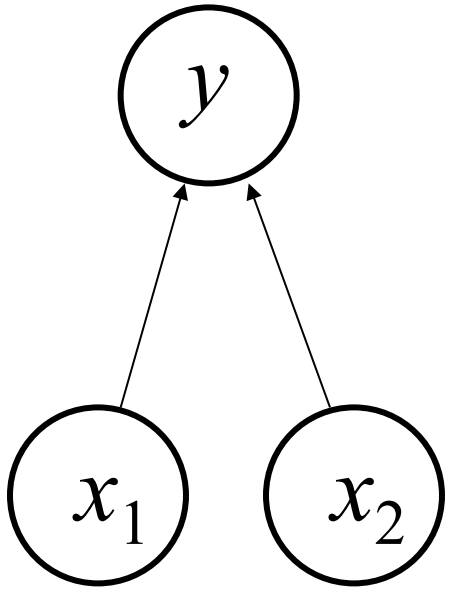
- Three student presentations:
  1. Lake et al 2015 by Ruijia
  2. Linzen et al by Elijah

Short rest 2

3. Schulz Buschoff et al by Khailanii

Perceptrons

+1 = cat, -1 = dog



perceptual features

The simplest kind of “feed forward” neural network

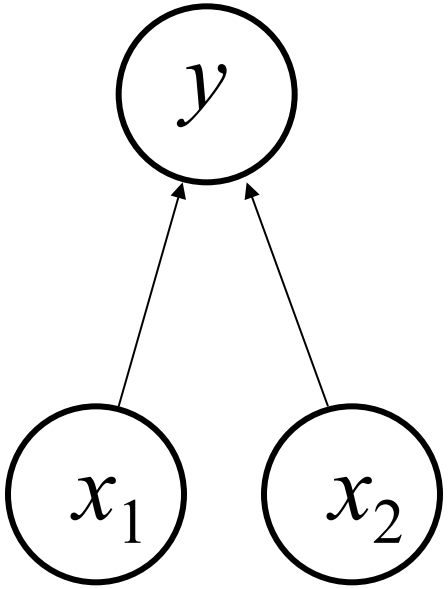
Originally introduced by Frank Rosenblatt (1958)

A linear classifier

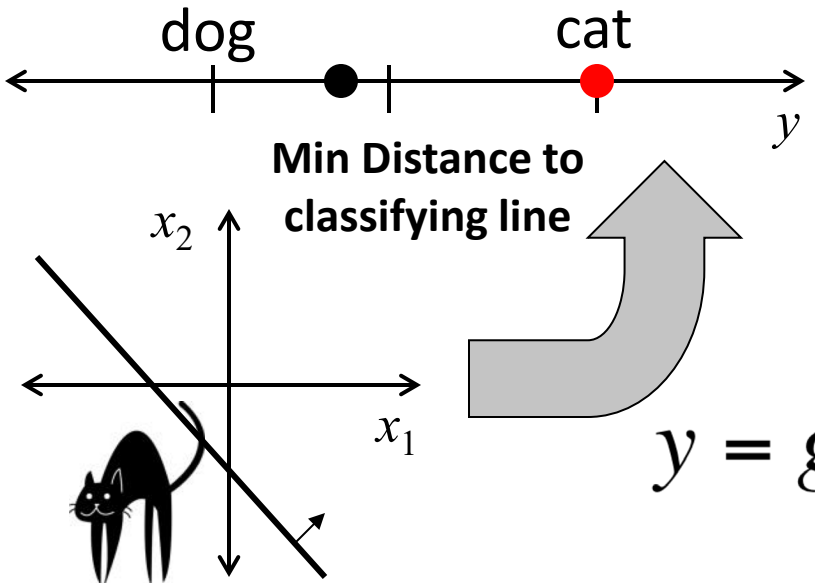
With just one output $\mathbf{W} = \begin{pmatrix} w_{11} & w_{12} \end{pmatrix}$

$g(\mathbf{Wx})$ will be constant for $\mathbf{x}$ orthogonal to $\mathbf{w}$

+1 = cat, -1 = dog



perceptual features

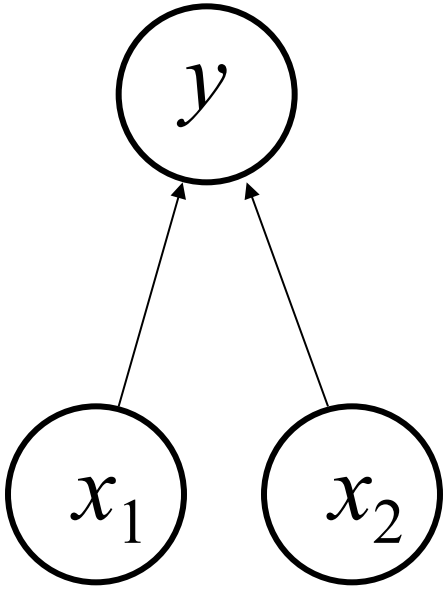


$y = g(\mathbf{Wx})$

Error-driven learning

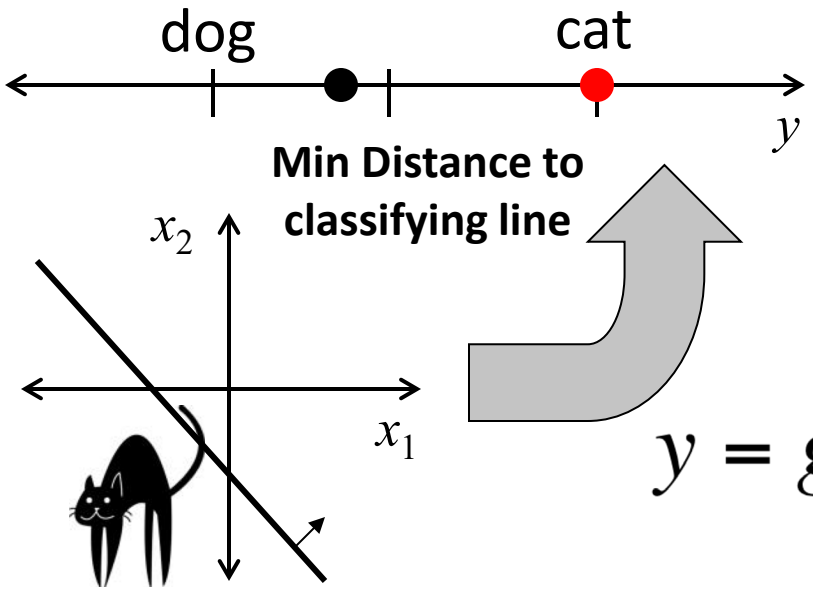
Define a measure of error in output, and find weights that minimize error

+1 = cat, -1 = dog



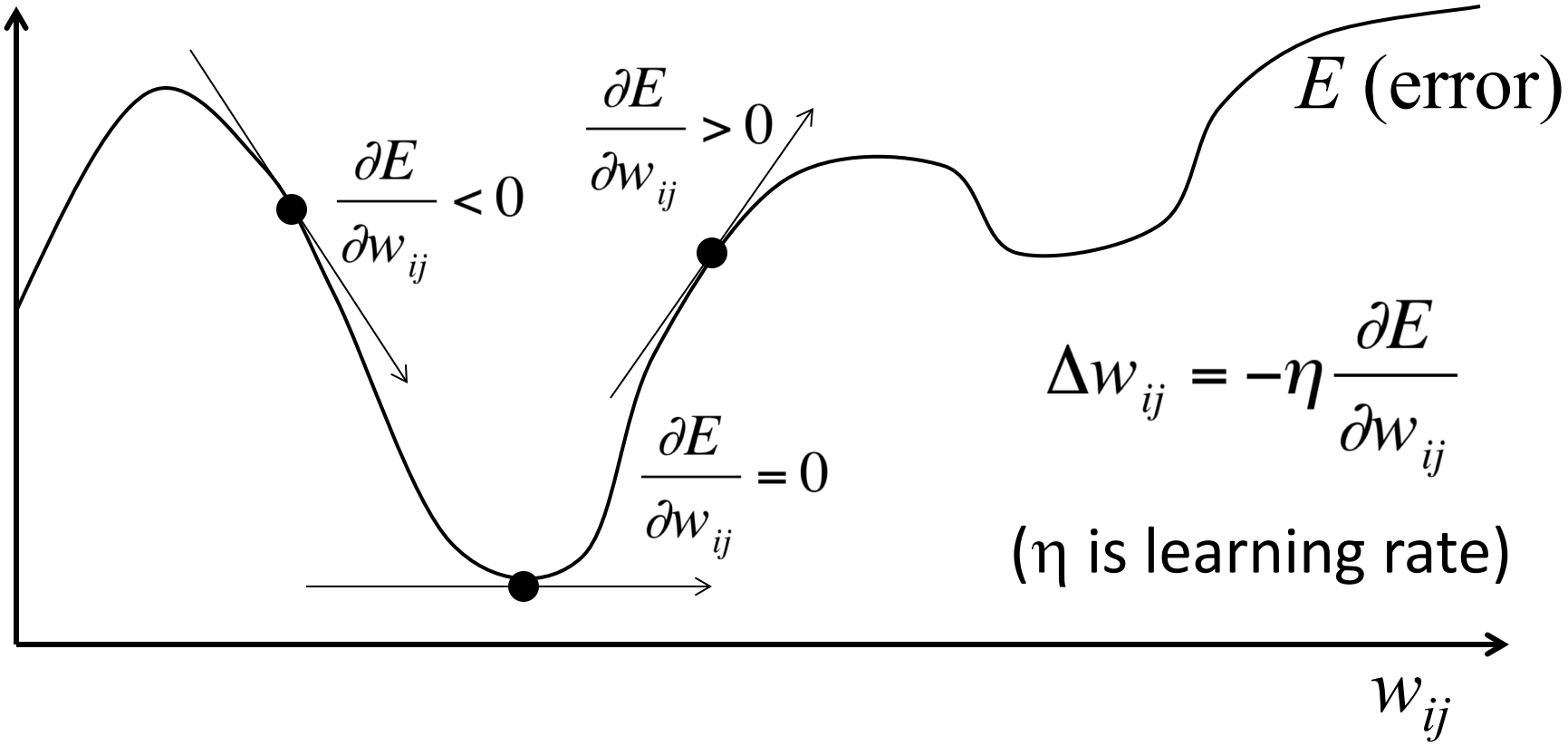
perceptual features

error: $E = \left(y - g(\mathbf{W}\mathbf{x})\right)^2$



$y = g(\mathbf{W}\mathbf{x})$

Gradient descent



A straightforward algorithm for finding a minimum:

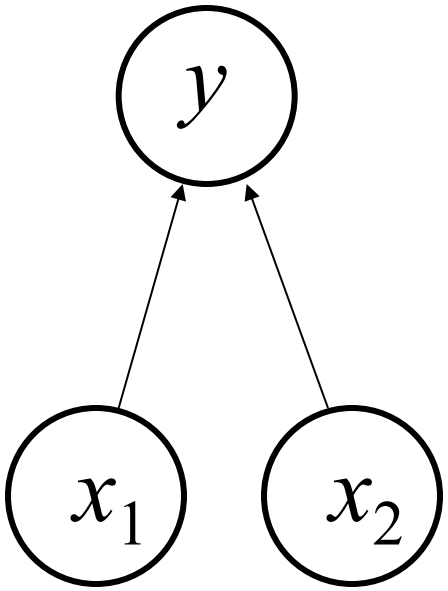
move in a direction where E decreases

stop if no such direction exists

With a linear output…

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$$

+1 = cat, -1 = dog



perceptual features

$$E = \left(y - g(\mathbf{W}\mathbf{x})\right)^2$$

$$E = \left(y - \mathbf{W}\mathbf{x}\right)^2$$

$$E = \left(y - (w_{11}x_1 + w_{12}x_2)\right)^2$$

$$\frac{\partial E}{\partial w_{ij}} = -2\left(y - (w_{11}x_1 + w_{12}x_2)\right)x_j$$

$$\Delta w_{ij} = \eta \left(y - \mathbf{W}\mathbf{x}\right) x_j$$

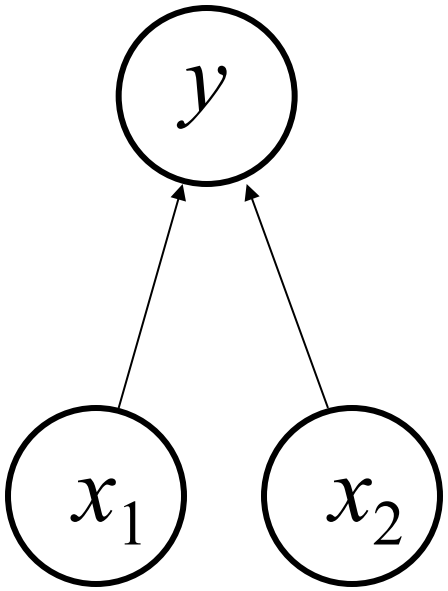
output error

influence of input

The Delta Rule

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$$

+1 = cat, -1 = dog



perceptual features

$$E = \left(y - g(\mathbf{W}\mathbf{x})\right)^2$$

for any function g with derivative g'

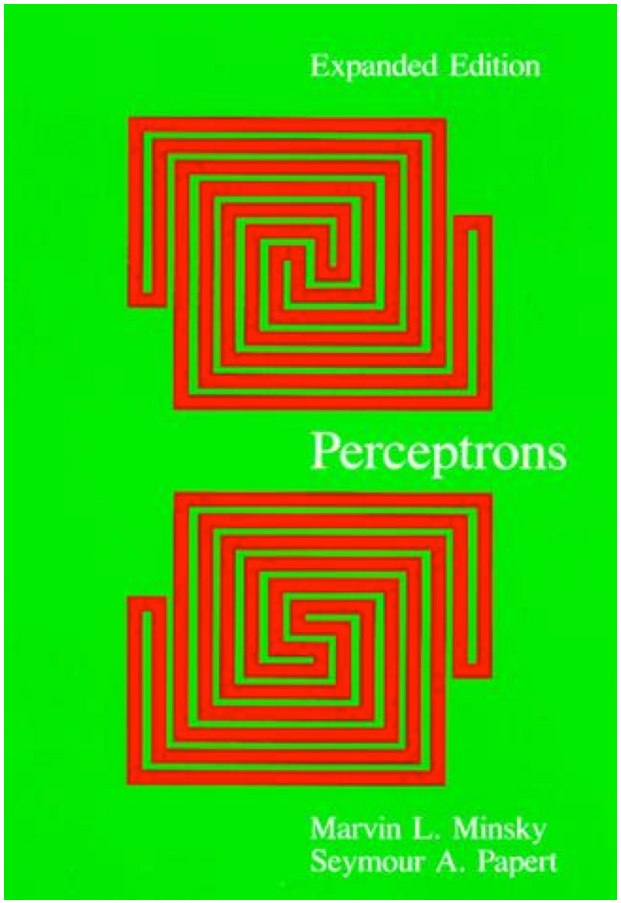
$$\frac{\partial E}{\partial w_{ij}} = -2\left(y - g(\mathbf{W}\mathbf{x})\right) g'(\mathbf{W}\mathbf{x})\, x_j$$

$$\Delta w_{ij} = \eta \left(y - g(\mathbf{W}\mathbf{x})\right) g'(\mathbf{W}\mathbf{x})\, x_j$$

output error

influence of input

Problems with simple networks

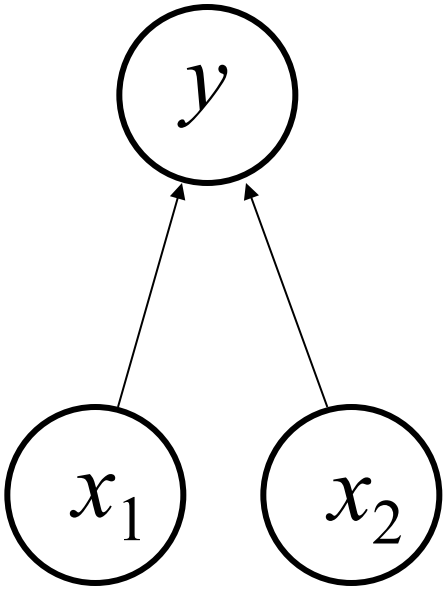


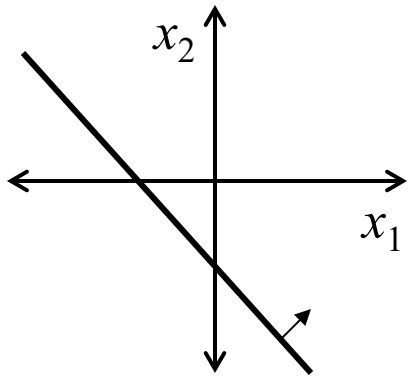
(1969)

Marvin Minsky

Seymour Papert

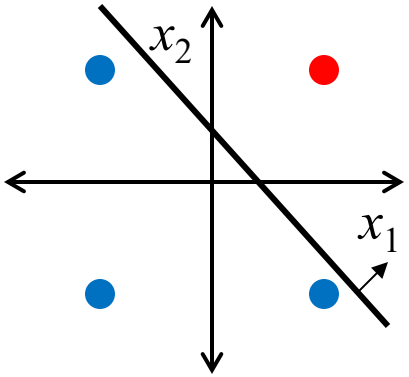
Problems with simple networks



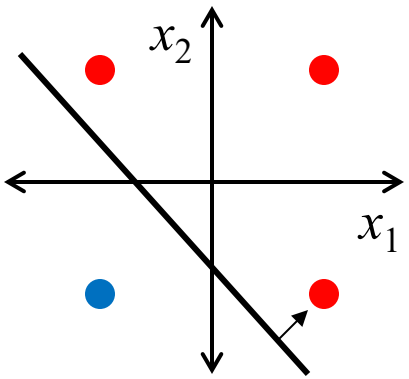


Some kinds of data are not linearly separable

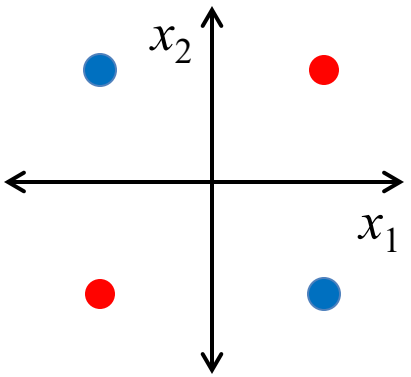
AND



OR



XOR



Problems with **linear** networks

Does adding layers help?

$$\mathbf{h} = f^{(1)}(\mathbf{x}; \mathbf{W}, \mathbf{c}) \qquad y = f^{(2)}(\mathbf{h}; \mathbf{w}, b)$$

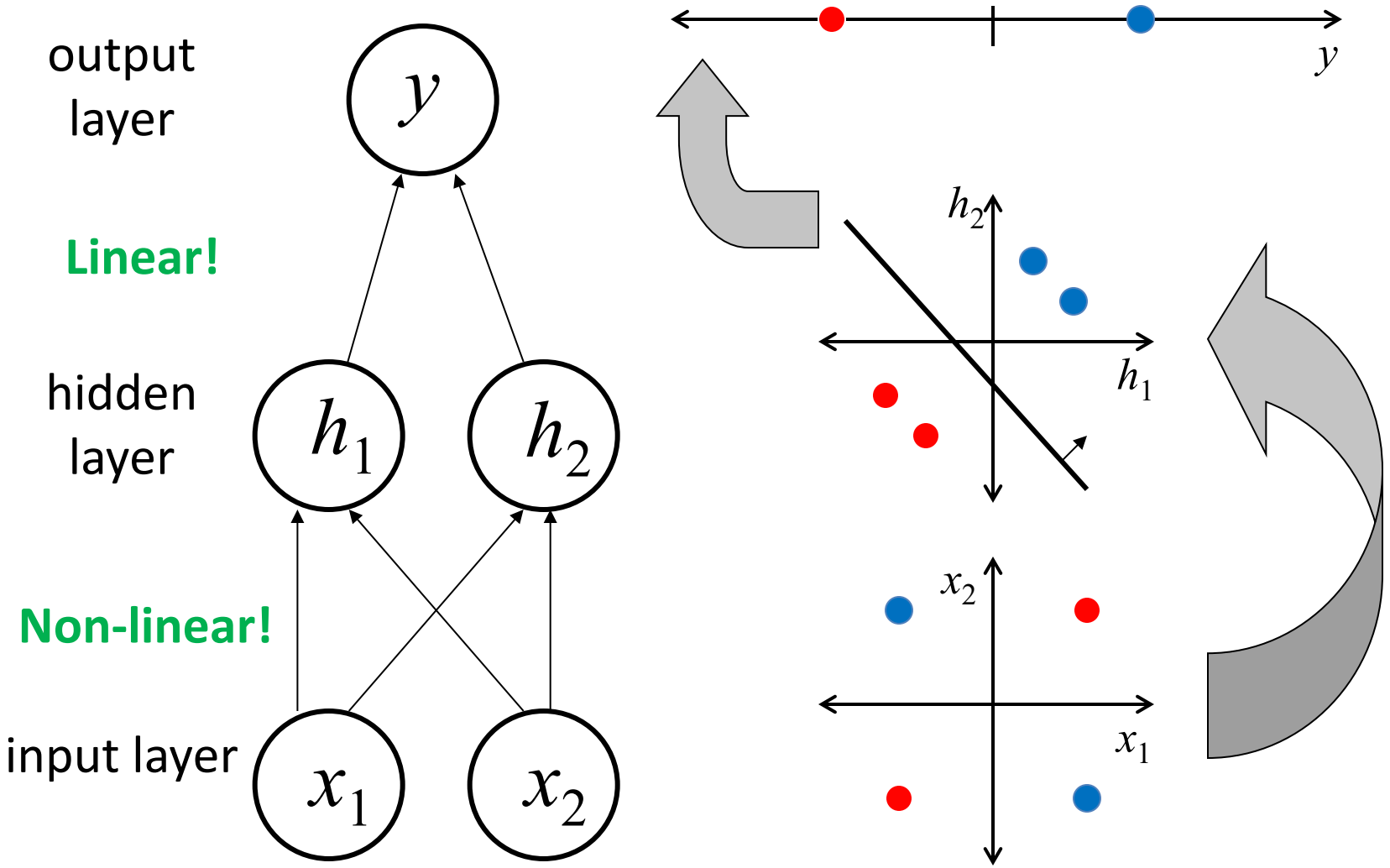
$$f(\mathbf{x}; \mathbf{W}, \mathbf{c}, \mathbf{w}, b) = f^{(2)}(f^{(1)}(x))$$

$$\mathbf{h} = f^{(1)}(\mathbf{x}) = \mathbf{W}^T\mathbf{x} \qquad f^{(2)}(\mathbf{h}) = \mathbf{h}^T\mathbf{w}$$

$$f(\mathbf{x}) = \mathbf{w}^T\mathbf{W}^T\mathbf{x} \Rightarrow f(\mathbf{x}) = \mathbf{x}^T\mathbf{w}'$$

$$\mathbf{w}' = \mathbf{W}\mathbf{w}$$

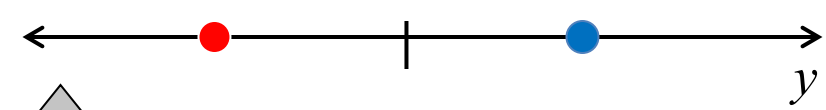
A solution: multiple layers + nonlinearity



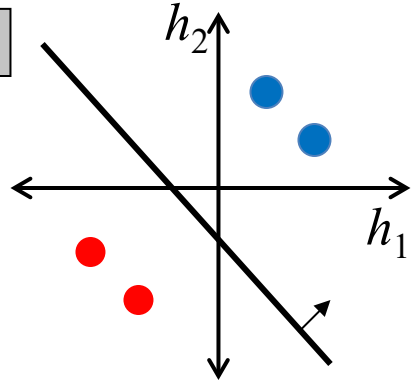
A solution: multiple layers

$$y = \mathbf{w}^T \max\{0, \mathbf{W}^T\mathbf{x} + \mathbf{c}\} + b \qquad b = 0$$

$$\mathbf{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \quad \mathbf{c} = \begin{bmatrix} 0 \\ -1 \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$$

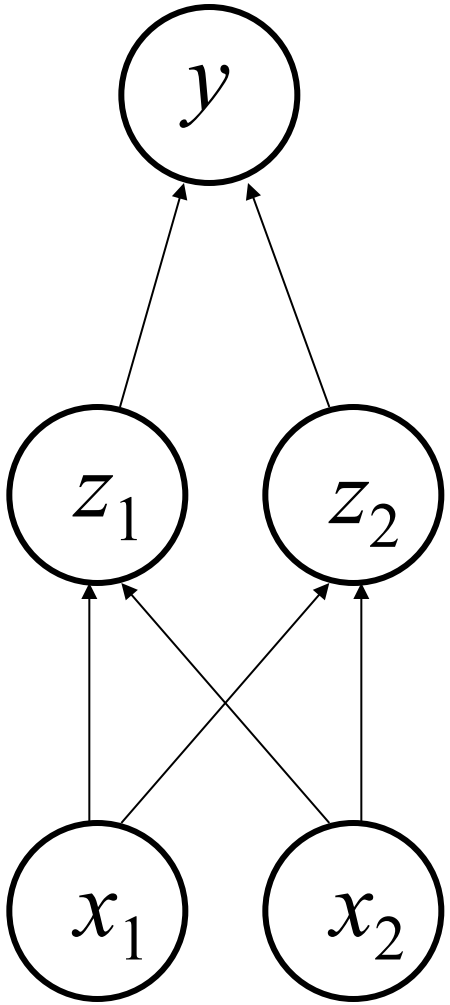


$$\mathbf{X} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix} \quad \mathbf{C}^T = \begin{bmatrix} 0 & -1 \\ 0 & -1 \\ 0 & -1 \\ 0 & -1 \end{bmatrix}$$



$$\mathbf{XW} = \begin{bmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{bmatrix} \quad \mathbf{XW} + \mathbf{C}^T = \begin{bmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}$$

$$\mathbf{w}^T \max(0, \begin{bmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}) = [1 \ -2] \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

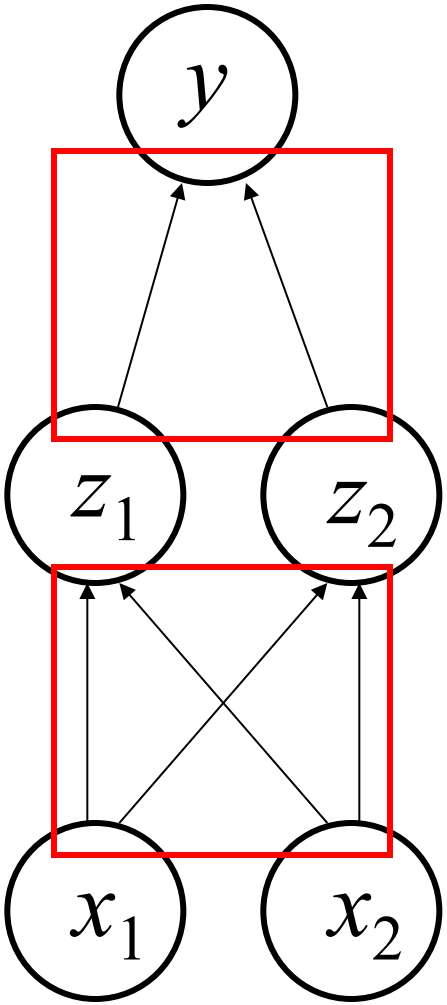


How do we learn the weights for a perceptron with multiple layers?

David Rumelhart

- Explored mathematical models of cognition
- One of the discoverers of backpropagation
  - widely used in cognitive and computer science
- Inspiration for the annual Rumelhart Prize

Training multi-layer networks



compute error

use delta rule

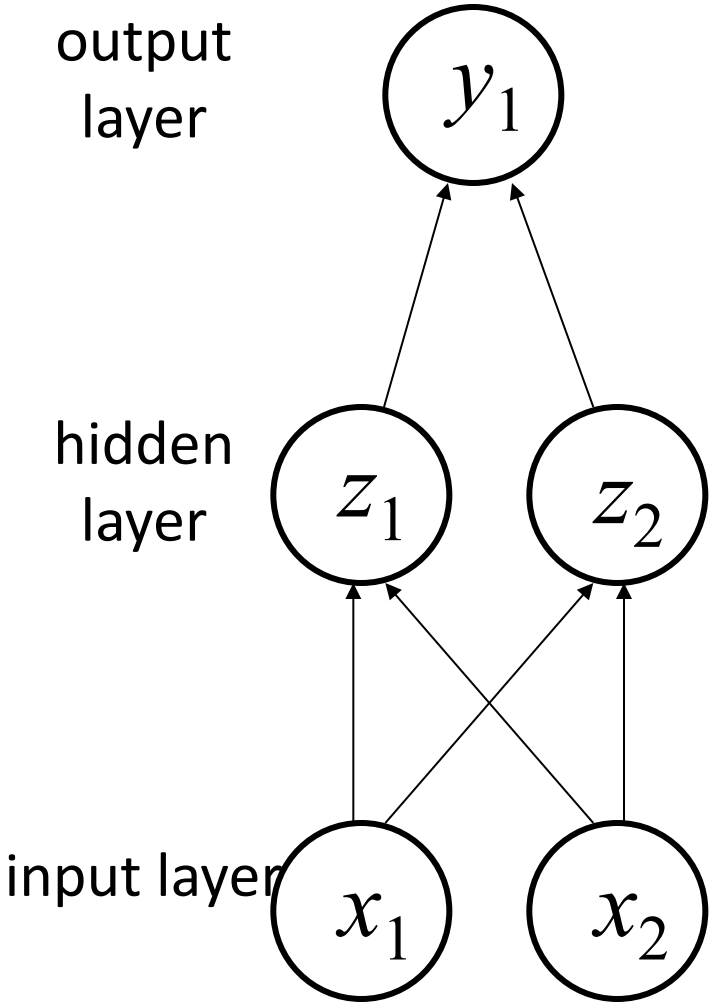
"propagate" error to hidden layer

use generalized delta rule (updating inputs according to their role in causing the hidden layer to have errors)

"Backpropagation"

(Rumelhart, Hinton, & Wilson, 1986)

Defining our terms



w_{ij}

w_{jk}

Backpropagation

(generalized delta rule)

Delta rule:

$$\Delta w_{ij} = \eta \left(y_i - g(input_i) \right) g'(input_i) \, z_j$$

Define a modified error term:

$$\Delta_i = \left(y_i - g(input_i) \right) g'(input_i)$$

We can write the delta rule as:

$$\Delta w_{ij} = \eta \, \Delta_i z_j$$

How do we work out the “error” for a hidden unit?

A little math

With two layers, we can write the error as

$$E = \left(y_i - g\left(\sum_j w_{ij} g(input_j) \right) \right)^2$$

where $input_j = \sum_k w_{jk} x_k$

and then apply the chain rule…

$$\frac{\partial E}{\partial w_{jk}} = -2\left(y_i - g\left(\sum_j w_{ij} g(input_j) \right) \right) g'\left(\sum_j w_{ij} g(input_j) \right) w_{ij} g'(input_j) x_j$$

Backpropagation
(generalized delta rule)

The appropriate error term is

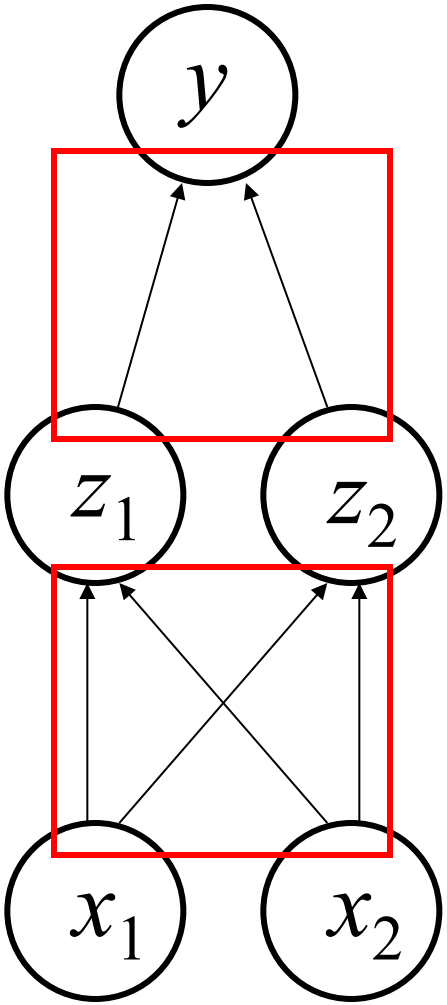
$$\Delta_j = \left(\sum_i w_{ij} \Delta_i \right) g'(input_j)$$

Each hidden unit is apportioned some blame!
Error is propagated "back" to hidden units…

We can write the generalized delta rule as:

$$\Delta w_{jk} = \eta \Delta_j x_k$$

Training multi-layer networks



compute error

use delta rule

propagate error to hidden layer

use generalized delta rule

"Backpropagation"

(Rumelhart, Hinton, & Wilson, 1986)

- Admin Matters
- Connectionism and neural network basics
- Linear neural networks
- Nonlinearity and Backpropagation
- Convolutional and Recurrent neural networks

Short rest 1

- Three student presentations:
  1. Lake et al 2015 by Ruijia
  2. Linzen et al by Elijah

Short rest 2

3. Schulz Buschoff et al by Khailanii

Convolutional Neural Networks

Convolutional Neural Networks are neural networks that use "convolution" instead of standard matrix multiplication.

Convolution

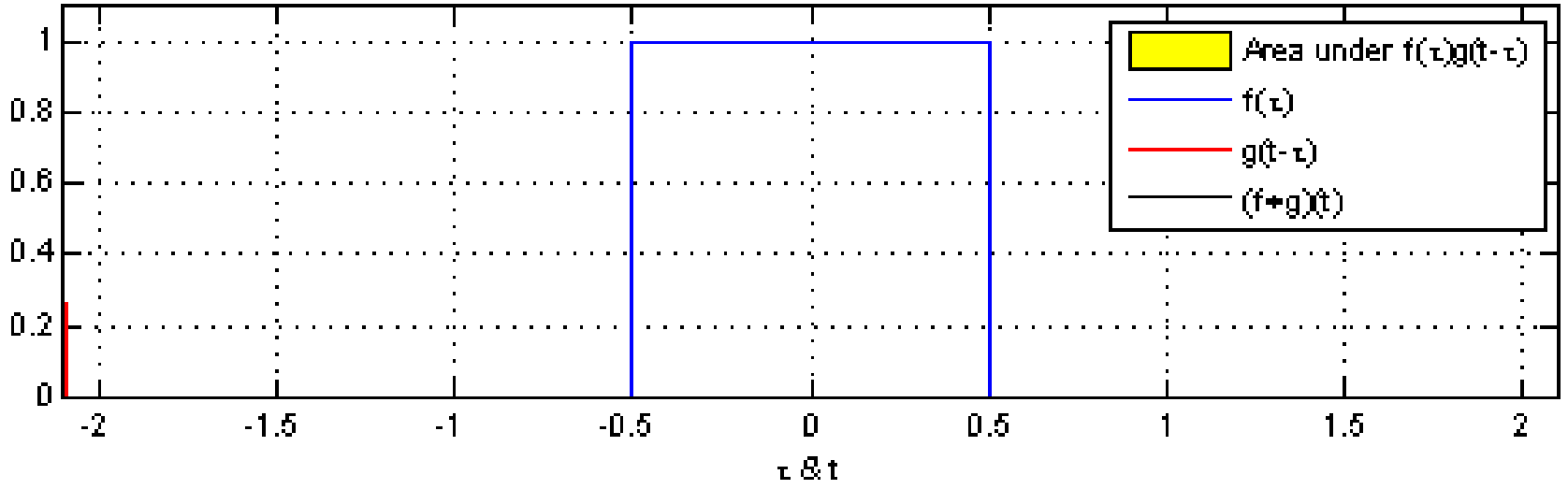
Say x(t) is the noisy sensor of the location of a spaceship
If we want the best estimate of the location at time *t*, we probably want to combine a few values.
Convolution is a way of doing this -- we could combine the weights at different times near *t*.

(larger values at *t* and decay as we go further away from *t*).

x is the input.
w is called the kernel (or sometimes a feature map/image)

$$s(t) = \int x(a)w(t-a)da = (x * w)(t)$$

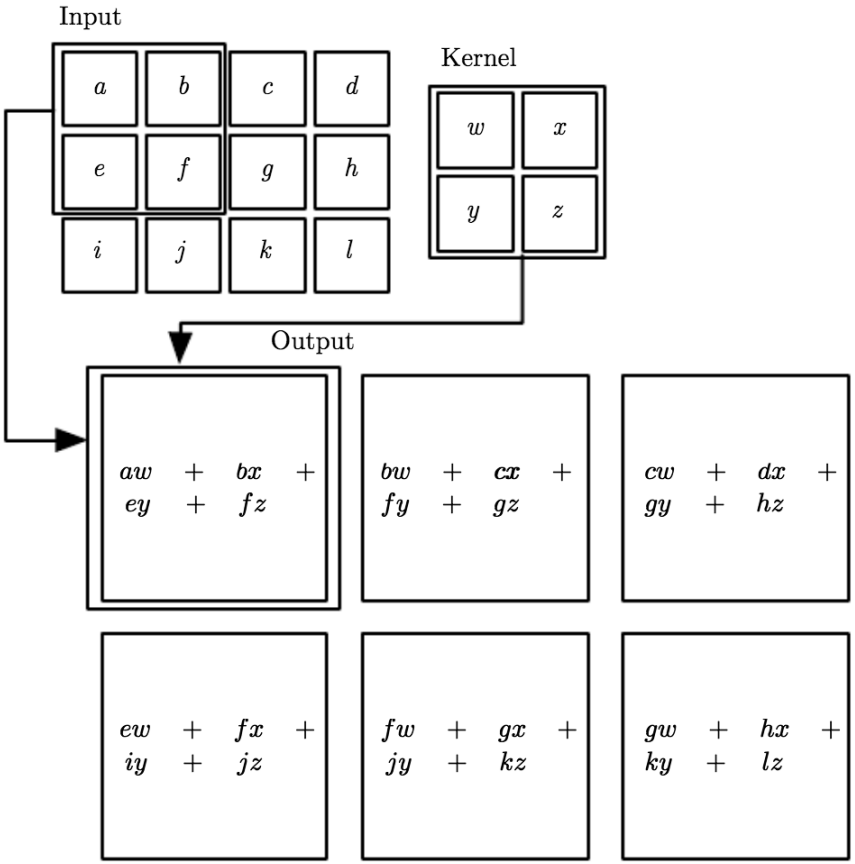
Moving vertical dotted line is current value of τ (a before)
Blue is the input x, written as f(τ) in this figure
Red is the kernel w as the integrand moving through with
width g(t- τ)
Yellow is current area of integral, f(τ)* g(t- τ)
Black is convolution (f * g)(t)



Convolutional Neural Networks

Convolutional Neural Networks are neural networks that use "convolution" instead of standard matrix multiplication.

They are often used in machine learning using images.



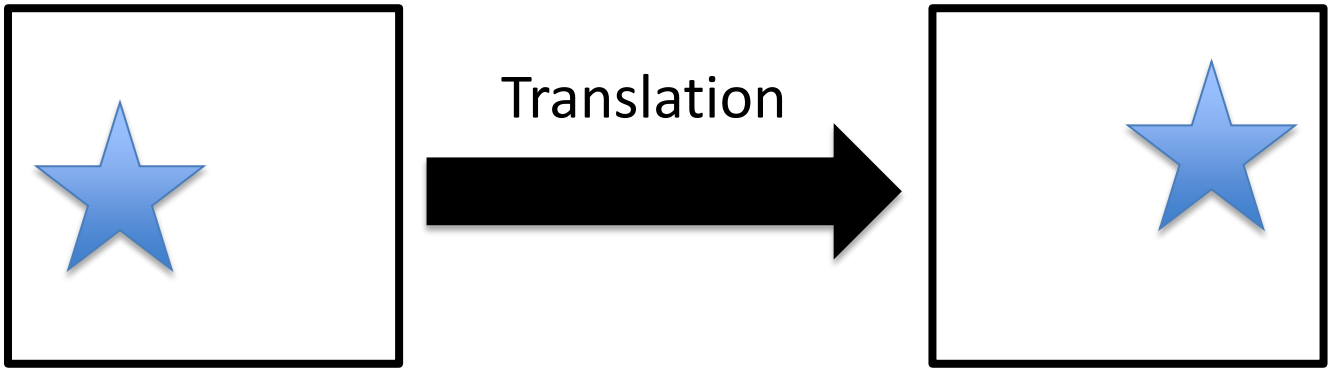
Convolutional Neural Networks

Convolutional Neural Networks are neural networks that use "convolution" instead of standard matrix multiplication.

They are often used in machine learning using images.

One interpretation is that you force multiple units that connect to different parts of the image to share the same weights.

It enables units that are **invariant** to types of transformation.



A different computational problem

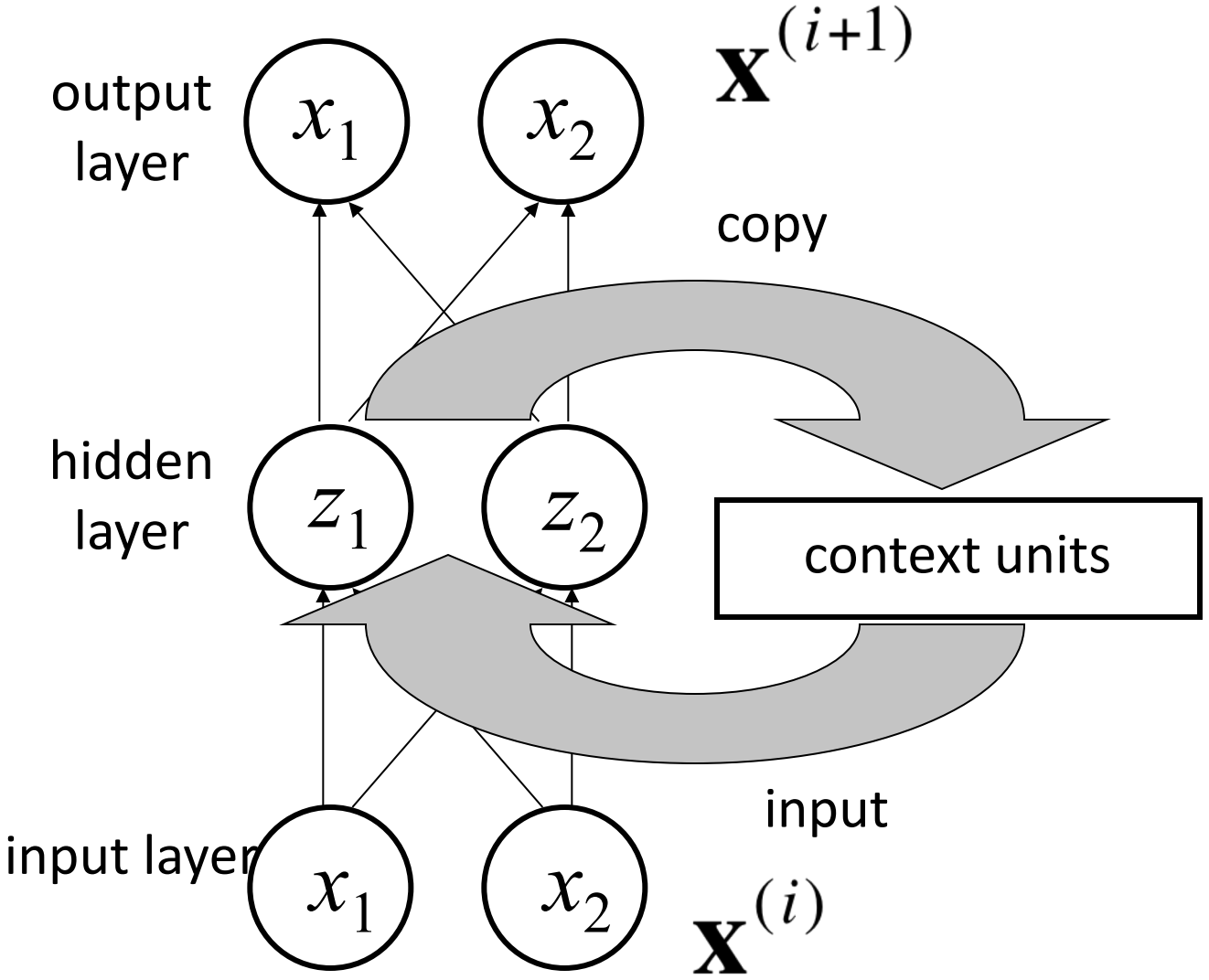
Learning language is a sequential prediction problem

A grammar is a function from previous words to the next word

Words are represented as points in space (vectors, distributed or local representations)

A grammar can be encoded as a set of weights in a neural network

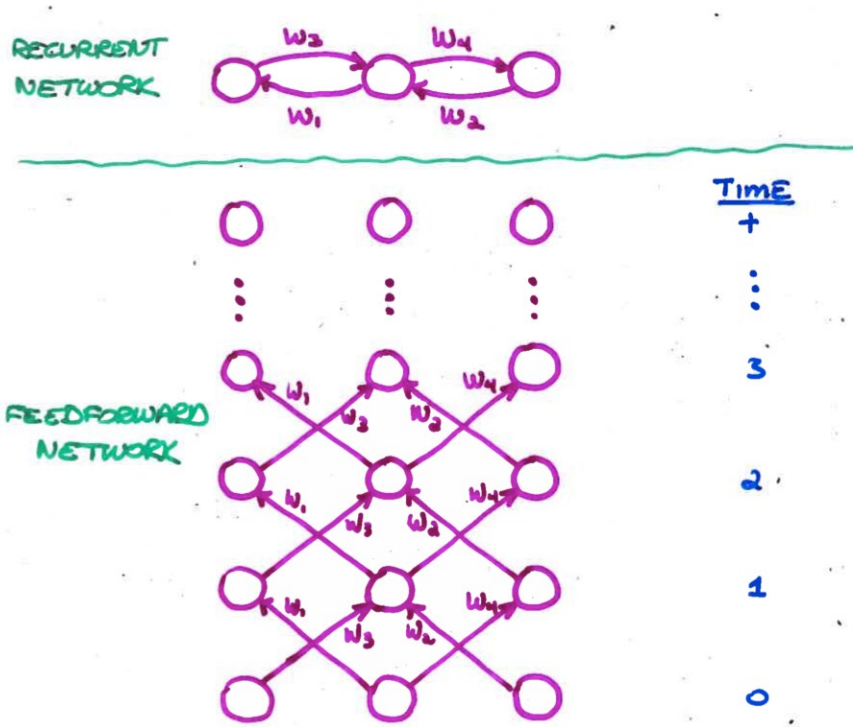
Simple recurrent networks



(Rumelhart et al., 1986; Elman, 1990)

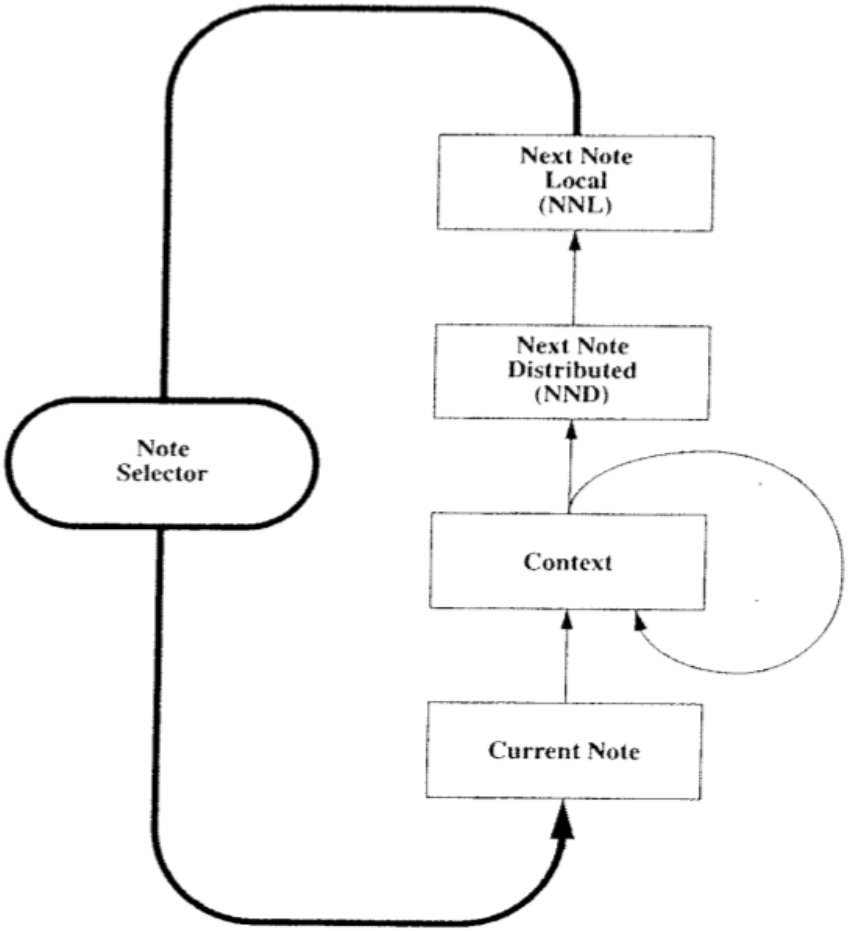
Unfolding A Recurrent Net

- Requires duplicat
  and weights)
  - one layer for ea
- Unfolded net has
  that of recurrent
  number of time s
- BP may violate th
  corresponding w
  - To enforce the
    set of underlyir
- Unfolding is nece
  learning.
  - Once weights a
    ordinary RNN



Slide from Michael Mozer

Neural Network Music Composition (Mozer, 1994)



Slide from Michael Mozer

Psychologically Grounded Representation Of Pitch

- Each pitch as point in 5 dimensional space

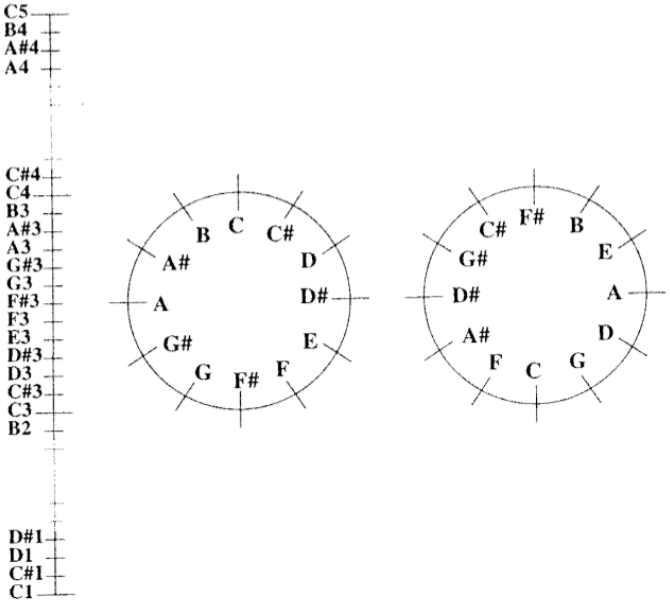
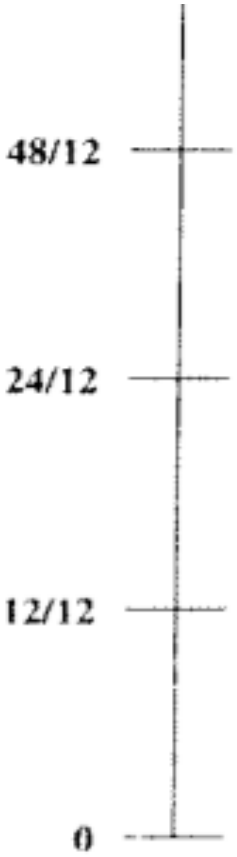
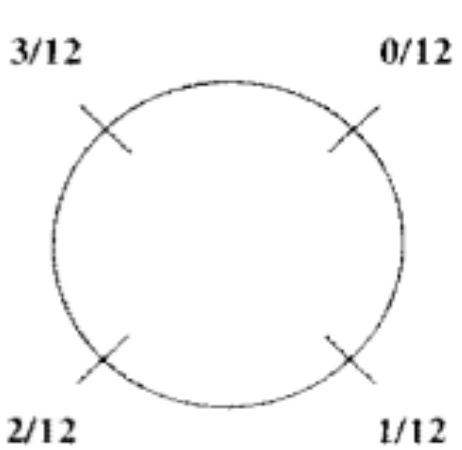


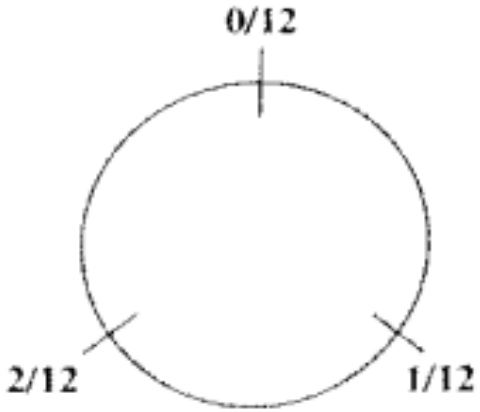
Figure 2. Shepard's (1982) pitch representation.

Slide from Michael Mozer

Analogous Representation Of Duration







Slide from Michael Mozer

Simulation Experiments

- All pieces transposed to common key
  - C major or A minor

- Bach (10 training examples)
- European folk melodies (25 examples)
- Waltzes (25 examples with harmonic accompaniment)
- Outcome
  - Network learns local structure but not global structure of musical organization

Slide from Michael Mozer